

Sergey Murylev, Yandex Ltd, <SergeyMurylev@gmail.com>, <smurylev@yandex-team.ru>
Anton Malakhov, Intel Corp., <anton.malakhov@intel.com>
Antony Polukhin, Yandex Ltd, <antoshkka@gmail.com>, <antoshkka@yandex-team.ru>

Date: 2017-06-14

Concurrent associative data structure with unsynchronized view

I. Introduction and Motivation

There's a lot of use-cases where a concurrent modification of unordered associative container is required. There were attempts to add such containers/data structures in the past ([N3425](#), [N3732](#), and others...)

This paper is another attempt to deal with the problem. This time we are trying to keep the interface familiar to users, hard to misuse but still functional.

Reference implementation is available at <https://github.com/BlazingPhoenix/concurrent-hash-map>.

II. Design decisions

A. Allow Open Addressing:

When users wish to use the concurrent associative data structure, they are searching for performance and scalability. Fastest known implementations rely on the **open addressing** [MemC3.pdf](#), so interface of this proposal allows to have Open Addressing implementation under the hood.

B. No functions that are easy to misuse:

In C++17 `std::shared_ptr::use_count()` function was removed because users were misusing it. Users were hoping that the result of the function is actual at the point where they were trying to use it. However as soon as the result is returned from the function it could expire as someone modifies the value from other thread.

We followed the C++17 decision and **removed all the functions that are useless/dangerous** in concurrent environments: `size()`, `count()`, `empty()`, `buckets_count()` and so forth.

C. No iterators:

Iterators must take care of synchronization, otherwise the user can not dereference the iterator at all. If Iterators do some synchronization it **affects performance**. Otherwise, if Iterators do some synchronization then **deadlocks will happen**. For example if in first thread we iterate from beginning to the end of the container and in the second thread we iterate from the end to the beginning, then the deadlock will happen in the middle as the iterators met.

It is possible to drop the open addressing idea and make the iterators to have shared ownership of buckets. In that case iterator may keep the bucket alive. This seems implementable and usable by users, but significantly **affects performance** by adding multiple additional atomic operations and adding additional indirections. We tried to stick to this idea for a long time and minimize the performance impact. Finally we created a list of use-cases for concurrent associative data structures and found out that in all of those use-cases iterators are useless or kill performance of the whole class (so are also useless). Instead of that, we came up with an idea of **unsynchronized view/policy**.

D. Unsynchronized view/policy with a full interface:

This is the **killer feature** of this proposal that attempts to fix all the limitations from above and provide a much more useful interface.

The idea is following: multiple operations on unordered containers make sense only if that container is not concurrently modified. User may take the responsibility that no-one is modifying the container at this point and gain all the operations and

iterators.

Such approach allows to initialize/prepare the data for container **without additional synchronization overhead**. It also **allows advanced usage**:

- Multiple threads use the same `concurrent_unordered_map` for reads and writes simultaneously.
- Multiple threads use `const` `unsynchronized_view` on the same `concurrent_unordered_map` simultaneously (no modifications through `concurrent_unordered_map` interface are allowed!).
- Single thread uses `unsynchronized_view` (no modifications are allowed from other threads!).
- Multiple threads use the same `const` `concurrent_unordered_map` for reads and multiple threads use `const` `unsynchronized_view` on the same `concurrent_unordered_map` simultaneously (ineffective: use multiple `const` `unsynchronized_view` instead).

E. No node operations, different from `std::unordered_map` iterator invalidation:

This is a consequence of allowing the open addressing as an underlying implementation.

F. Allow element visitation using the synchronization of the container:

This is a very risky decision because it unleashes new ways for deadlocking/breaking the container (users may recursively visit the same value; users may call heavy functions that will degrade the overall performance of the container; users can call some parallel functions of the standard library that may potentially use the same mutexes as the container implementation...).

However, there's a lot of use-cases where a value must be updated depending on value that is in the container. Without a visitation, there's no way to do that safely, as all the functions return values by copy. See examples [B](#) and [C](#).

III. Draft interface and informal description

`concurrent_unordered_map` class is an associative data structure that provides an effective key-value storage and does not meet the unordered associative container requirements. Concurrent member functions calls on the same instance of `concurrent_unordered_map` have well defined behavior.

```
namespace std {
    template <class Key, class T, class Hasher, class Equality, class Allocator>
    class unsynchronized_view;

    template <class Key,
              class T,
              class Hasher = hash<Key>,
              class Equality = equal_to<Key>,
              class Allocator = allocator<pair<const Key, T>> >
    class concurrent_unordered_map {
    public:
        // types:
        using key_type           = Key;
        using mapped_type        = T;
        using value_type         = pair<const Key, T>;
        using hasher              = Hasher;
        using key_equal           = Equality;
        using allocator_type      = Allocator;
        using size_type           = implementation-defined;

        // construct/destroy:
        concurrent_unordered_map() noexcept;
        explicit concurrent_unordered_map(size_type n,
                                         const Hasher& hf = hasher(),
                                         const key_equal& eql = key_equal(),
                                         const allocator_type& a = allocator_type());

        template <typename InputIterator>
        concurrent_unordered_map(InputIterator first, InputIterator last,
                                size_type n = implementation-defined,
                                const hasher& hf = hasher(),
                                const key_equal& eql = key_equal(),
                                const allocator_type& a = allocator_type());

        concurrent_unordered_map(const Alloc&);
        concurrent_unordered_map(concurrent_unordered_map&&) noexcept;
        concurrent_unordered_map(concurrent_unordered_map&&, const Allocator&);
        concurrent_unordered_map(initializer_list<value_type> il,
                                size_type n = implementation-defined,
                                const hasher& hf = hasher(),
                                const key_equal& eql = key_equal(),
                                const allocator_type& a = allocator_type());
        ~concurrent_unordered_map();

        // unsynchronized view/policy:
        unsynchronized_view<Key, T, Hasher, Equality, Allocator> lock_table() const noexcept;

        // concurrent-safe assignment:
        concurrent_unordered_map& operator=(concurrent_unordered_map&&) noexcept;
```

```

concurrent_unordered_map& operator=(initializer_list<value_type> il);

// executes `f` under a `lock` if `key` is in *this.
template <typename F>
void visit(const key_type& key, F&& f);

// executes `f` under a `lock`. If `key` is not in *this calls emplace(std::forward<Args>(args)...) first.
template <typename F, typename... Args>
void emplace_or_visit(const key_type& key, F&& f, Args&&... args);

void swap(concurrent_unordered_map&)
noexcept(
    allocator_traits<Allocator>::is_always_equal::value &&
    is_nothrow_swappable_v<Hasher> &&
    is_nothrow_swappable_v<Pred>);
void clear() noexcept;

// concurrent-safe element retrieval:
optional<mapped_type> find(const key_type& key) const;
mapped_type find(const key_type& key, const mapped_type& default_value) const;

// concurrent-safe modifiers:
template <typename... Args>
bool emplace(const key_type& key, Args&&... val);
template <typename... Args>
bool emplace(key_type&& key, Args&&... val);

bool insert(const value_type& x);
bool insert(value_type&& x);
template<class InputIterator>
void insert(InputIterator first, InputIterator last);

template <typename V>
bool insert_or_assign(const key_type& key, V&& val);
template <typename V>
bool insert_or_assign(key_type&& key, V&& val);

optional<value_type> extract(const key_type& x);

size_type update(const key_type& key, const value_type& val);
size_type update(const key_type& key, value_type&& val);

size_type erase(const key_type& key);

template<class H2, class P2>
void merge(concurrent_unordered_map<Key, T, H2, P2, Allocator>& source);
template<class H2, class P2>
void merge(concurrent_unordered_map<Key, T, H2, P2, Allocator>&& source);
// Note: merge for concurrent_unordered_multimap?
};

```

unsynchronized_view class refers concurrent_unordered_map and provides a concurrent unsafe interface to concurrent_unordered_map that satisfies unordered associative container requirements (except iterator invalidation requirements, load_factor functions, size() complexity requirements and node operations).

[*Note:* Concurrent non const member functions calls on the instances of unsynchronized_view referencing the same concurrent_unordered_map are not safe! - *end note.*]

[*Note:* Concurrent member functions calls on the concurrent_unordered_map instance A and on the unsynchronized_view referencing the instance A are not safe! - *end note.*]

```

template <class Key, class T, class Hasher, class Equality, class Allocator>
class unsynchronized_view {
    concurrent_unordered_map<Key, T, Hasher, Equality, Allocator>& delegate; // exposition only

public:
    // types:
    using key_type          = Key;
    using mapped_type       = T;
    using value_type        = pair<const Key, T>;
    using hasher            = Hasher;
    using key_equal         = Equality;
    using allocator_type    = Allocator;

    using pointer           = typename allocator_traits<Allocator>::pointer;
    using const_pointer     = typename allocator_traits<Allocator>::const_pointer;
    using reference         = value_type&;
    using reference         = const value_type&;
    using size_type         = implementation-defined;
    using difference_type   = implementation-defined;
    using iterator          = implementation-defined;
    using const_iterator    = implementation-defined;
    using local_iterator    = implementation-defined;
    using const_local_iterator = implementation-defined;

    // construct/copy/destroy:
    unsynchronized_view() = delete;
    unsynchronized_view(concurrent_unordered_map<Key, T, Hasher, Equality, Allocator>& delegate);
    unsynchronized_view(const unsynchronized_view&) noexcept = default;
    unsynchronized_view& operator=(const unsynchronized_view&) noexcept = default;

```

```

~unsynchronized_view() = default;

// iterators:
iterator      begin() noexcept;
const_iterator begin() const noexcept;
iterator      end() noexcept;
const_iterator end() const noexcept;
const_iterator cbegin() const noexcept;
const_iterator cend() const noexcept;

// capacity:
bool empty() const noexcept;
size_type size() const noexcept;
size_type max_size() const noexcept;

// modifiers:
template<typename... Args>
pair<iterator, bool> emplace(Args&&... args);
template<typename... Args>
iterator emplace_hint(const_iterator hint, Args&&... args);

pair<iterator, bool> insert(const value_type& x);
pair<iterator, bool> insert(const value_type&& x);
template<class P> pair<iterator, bool> insert(P&& x);
iterator insert(const_iterator hint, const value_type& x);
iterator insert(const_iterator hint, const value_type&& x);
template<class P> pair<iterator, bool> insert(const_iterator hint, P&& x);
template<class InputIterator> void insert(InputIterator first, InputIterator last);
void insert(initializer_list<value_type> il);

template <typename... Args>
pair<iterator, bool> try_emplace(const key_type& k, Args&&... args);
template <typename... Args>
pair<iterator, bool> try_emplace(key_type&& k, Args&&... args);
template <typename... Args>
iterator try_emplace(const_iterator hint, const key_type& k, Args&&... args);
template <typename... Args>
iterator try_emplace(const_iterator hint, key_type&& k, Args&&... args);
template <class M>
pair<iterator, bool> insert_or_assign(const key_type& k, M&& obj);
template <class M>
pair<iterator, bool> insert_or_assign(key_type&& k, M&& obj);
template <class M>
iterator insert_or_assign(const_iterator hint, const key_type& k, M&& obj);
template <class M>
iterator insert_or_assign(const_iterator hint, key_type&& k, M&& obj);

iterator erase(iterator position);
iterator erase(const_iterator position);
size_type erase(const key_type& k);
iterator erase(const_iterator first, const_iterator last);
void swap(unordered_map&& other);
noexcept(allocator_traits<Allocator>::is_always_equal::value &&
is_nothrow_swappable_v<Hasher> &&
is_nothrow_swappable_v<Pred>);
void clear() noexcept;

template<class H2, class P2>
void merge(concurrent_unordered_map<Key, T, H2, P2, Allocator>&& source);
template<class H2, class P2>
void merge(concurrent_unordered_multimap<Key, T, H2, P2, Allocator>&& source);
// Note: merge for concurrent_unordered_multimap ?

// observers:
allocator_type get_allocator() const;
hasher hash_function() const;
key_equal key_eq() const;

// map operations:
iterator find(const key_type& k);
const_iterator find(const key_type& k) const;
size_type count(const key_type& k) const;
pair<iterator, iterator> equal_range(const key_type& k);
pair<const_iterator, const_iterator> equal_range(const key_type& k) const;

// element access:
mapped_type& operator[] (const key_type& k);
mapped_type& operator[] (key_type&& k);
const mapped_type& at(const key_type& k) const;
mapped_type& at(const key_type& k);

// bucket interface:
size_type bucket_count() const;
size_type max_bucket_count() const;
size_type bucket_size(size_type n);
size_type bucket(const key_type& k) const;
local_iterator begin(size_type n);
const_local_iterator begin(size_type n) const;
local_iterator end(size_type n);
const_local_iterator end(size_type n) const;
const_local_iterator cbegin(size_type n) const;
const_local_iterator cend(size_type n) const;

// hash policy:

```

```

    void rehash(size_type n);
};
}

```

[*Note*: Following use-cases are allowed:

- Multiple threads use the same `concurrent_unordered_map` for reads and writes simultaneously
- Multiple threads use `const` `unsynchronized_view` on the same `concurrent_unordered_map` simultaneously (no modifications through `concurrent_unordered_map` interface are allowed!)
- Single thread uses `unsynchronized_view` (no modifications are allowed from other threads!)
- Multiple threads use the same `const` `concurrent_unordered_map` for reads and multiple threads use `const` `unsynchronized_view` on the same `concurrent_unordered_map` simultaneously (ineffective: use multiple `const` `unsynchronized_view` instead).

- *end note*.]

IV. Some usage examples in pseudo code

A. User session cache

```

#include <concurrent_hash_map>
#include <string_view>
#include <memory>

using namespace std;
void precess_user(string_view name, shared_ptr<const user_t> user);
auto get_new_user();
auto get_request();

int main() {
    concurrent_unordered_map<string_view, shared_ptr<user_t> > users;

    // single threaded fill
    read_users_from_file(users.lock_table())

    constexpr unsigned threads_count = 10;
    while(1) {
        // concurrent work:
        std::atomic<int> b{threads_count * 100500};
        thread threads[threads_count];

        for (auto& t: threads) {
            // processing users
            t = thread([&users, &b]() {
                while (--b > 0) {
                    auto [user_name, data] = co_await get_request();
                    auto u = users.find(user_name);
                    if (!u) continue;

                    shared_ptr<const user_t> user = *u;
                    precess_user(user, data);
                }
            });
        }

        // accepting users
        while (--b > 0) {
            auto [new_user_name, user] = co_await get_new_user();
            users.insert(new_user_name, user);
        }

        for (auto& t: threads) {
            t.join();
        }

        // single threaded processing:
        auto unsafe_users = users.lock_table();
        count_statistics(unsafe_users);
        dump_to_file(unsafe_users);
        cleanup(unsafe_users);
    }
}

```

B. Unique events processor/events deduplicator

```

#include <concurrent_hash_map>
#include <algorithm>

int main() {
    using namespace std;
    using event_id = ...;
    struct event_data {
        event_data(const event_data&) = delete;
        event_data& operator=(const event_data&) = delete;
    };
}

```

```

}; ...

concurrent_unordered_map<event_id, unique_ptr<event_data> > events;

// Getting unique events.
auto event_generators = get_event_generators();
for_each(execution::par_unseq, event_generators.begin(), event_generators.end(), [&events](auto& g) {
    while (1) {
        auto [event_name, data] = co_await g.get_event();
        if (event_name.empty()) {
            break; // no more events
        }

        events.emplace_or_visit(event_name, [&data](unique_ptr<event_data>& v){
            if (v || v->priority() < data->priority()) {
                std::swap(data, v);
            }
        }, unique_ptr<event_data>{});
    }
});

auto v = events.lock_table();
for_each(execution::par_unseq, v.begin(), v.end(), [](auto& e) {
    process(e.first, std::move(e.second));
});
}

```

C. Gathering statistics

```

#include <concurrent_hash_map>
#include <utility>

int main() {
    using namespace std;
    using id_t = ...;
    using use_count_t = size_t;

    concurrent_unordered_map<id_t, use_count_t> stats;

    constexpr unsigned threads_count = 10;
    thread threads[threads_count];
    for (auto& t: threads) {
        t = thread([&stats]() {
            while (1) {
                auto [id, data] = co_await get_something();
                stats.emplace_or_visit(
                    id,
                    [](auto& v){ ++v; },
                    0
                );

                precess_stuff(id, data);
            }
        });
    }

    for (auto& t: threads) {
        t.join();
    }

    process_stats(events.lock_table());
}

```

D. HFT Order book

```

#include <concurrent_hash_map>
#include <unordered_map>
#include <string_view>

using namespace std;

struct orders_book {
    using order_key = string;
    using order_key_ref = string_view;
    enum class order_amount: std::size_t {};
    enum class order_type: std::size_t {};
    using order_t = std::pair<order_type, order_amount>;

private:
    concurrent_unordered_map<order_key, order_t > orders;

    // For debug only: optimistic attempt to catch concurrent use of snapshot() function.
#ifdef NDEBUG
    atomic<size_t> uses_;
    struct in_use {
        atomic<size_t>& uses_;

        in_use(atomic<size_t>& uses)
            : uses_(uses)
        {

```

```

        ++ uses_;
    }

    ~in_use() {
        -- uses_;
    }
};

in_use set_in_use() noexcept {
    return {uses_};
}
void validate() const {
    const size_t v = uses_;
    assert(v == 0);
}
#else
struct in_use {};
static in_use set_in_use() noexcept { return {}; }
static void validate() noexcept {}
#endif

orders_book(const orders_book&) = delete;
orders_book& operator=(const orders_book&) = delete;

public: // Therad safe:
orders_book() = default;

void place(order_key_ref o, order_t&& v) {
    const auto guard = set_in_use();
    orders_.insert_or_assign(o, std::move(v));
}

auto try_get(order_key_ref o) const {
    return orders_.find(o);
}

auto try_bet(order_key_ref o) {
    const auto guard = set_in_use();
    return orders_.erase(o);
}

public: // Not thread safe!
unordered_map<order_key, order_t > snapshot() const {
    validate();
    auto v = orders_.lock_table();
    validate();
    unordered_map<order_key, order_t > ret{v.begin(), v.end()};
    validate();
    return ret;
}
};

```